

ESP32+W5500 12V電源/PTT制御 開発仕様書

本書はShonan_Lite-RasPi5の同名文書(同じ基板・同じファームウェア)を、Shonan_lite-PI4向けに連携部分を置き換えたものである。

英語版 / English version: W5500_PA_PTT_Control_spec_en.md

項目	内容
版数	Rev.2.3
作成日	2026-08-07 (Rev.2.0更新: 2026-08-30、実機回路(KiCad)に合わせて全面改訂／Rev.2.1更新: 2026-08-30、電源系統を訂正。J3(外付けDCDCバックコンバータ)は廃止し、U2(L7805)の+5VをMCU1とU1(TA48033S)の両方に供給する単一系統に修正／Rev.2.2更新: 2026-08-31、J1をFreenove実機の40pin DevKitCソケット配列に修正しMCU1もJ1と同じピン番号・信号名に統一。Power(スイッチ後12V)系統に赤色LED(D1)、+12V(入力側)系統に緑色LED(D2)の表示回路を追加／Rev.2.3更新: 2026-09-30、R8を100Ωに変更(KiCad回路図に合わせる)しR9を追記、IPアドレスの記述を固定IP方式に統一、12V電源ONまでの遅延を実装値(5秒)に訂正、Shonan_lite-PI4側の「ESP32 W5500を使用する」設定とPi4 GPIO21によるPTT出力を追記)
対象ボード	ESP32 (WROVER系、無印ESP32) + W5500 イーサネットモジュール
対象スケッチ	hardware/W5500_PA_PTT_Control/W5500_PA_PTT_Control.ino

連携先	shonan-android (DATV送信アプリ)、Shonan_lite-PI4 (pi4/gui、送信画面のTX開始/終了およびアプリ起動/終了に連動)
ステータス	実機ESP32(ESP32-D0WD-V3)へ書き込み・起動確認済み (2026-08-30)。無線機を含めた実地テスト(12V電源/PTT駆動回路との結合試験)は未実施

1. 目的・スコープ

shonan-android の送信ボタン操作に連動して **PTT** を、Shonan_lite-PI4(pi4/gui)アプリの起動/終了操作に連動して **12V電源(PA等の外部機器用)** を、それぞれイーサネット経由でON/OFF制御する。

Rev.1.1まではLNA/PTT/PAの3ch・シーケンス制御(送信時にLNAを切り離してからPTT/PAを立ち上げる等)を想定していたが、実機回路(KiCad)ではLNA用の駆動回路が存在せず、**12V電源ON/OFF (2SJ334によるハイスайдスイッチ)**と**PTT ON/OFFの2機能のみ**が実装されている。本書はこの実機回路に合わせて全面改訂した。

スコープに含むもの

- ・ ESP32 + W5500 による12V電源/PTT 2チャンネルのON/OFF制御
- ・ shonan-androidからのHTTPリクエストによるPTTのTX/RX切替
- ・ Shonan_lite-PI4(pi4/gui)アプリの起動/終了に連動した12V電源のON/OFF
- ・ ブラウザによる手動確認・デバッグ用UI

スコープに含まないもの

- ・ shonan-androidアプリ側での送信ボタン実装・HTTPリクエスト送出处理(別途アプリ側での対応が必要)
- ・ 12V電源の供給先となるPA・LNA等自体の回路設計(利用者の無線機構成に依存)

2. ハードウェア構成

2.1 使用部品

部品	備考
ESP32 (WROVER モジュール等)	無印ESP32。ESP32-C3等のネイティブUSBチップとは別物
W5500 イーサネットモジュール	SPI接続。MACアドレス内蔵なしのためスケッチ内で任意設定
Q5 (2SJ334)	P-ch パワーMOSFET。12V電源のハイサイドスイッチ（旧リレーK3を置き換え）
Q1 (2SC1815)	Q5のゲート駆動用NPNトランジスタ（GPIO26でON/OFF）
R8 (100Ω)	Q5のゲートを+12Vへプルアップする抵抗（Q1 OFF時にQ5をOFFに保つ）
Q3 (2SC1815)	PTT_ON信号駆動用NPNトランジスタ（GPIO27でON/OFF、オープンコレクタ的にGND側へ落とす）
R9 (10kΩ)	W5500(A1)のRST(GPIO21と同一ネット)を+3V3_Aへプルアップする抵抗
J2 (DC_IN_13V8)	外部電源(13.8V/12V系)の入力コネクタ
U2 (L7805)	+12V→+5Vのリニアレギュレータ(TO-220)。生成した+5V(+5v0)はMCU1(J1)とU1の両方に供給される
U1 (TA48033S)	U2出力の+5V→+3.3Vのリニアレギュレータ(TO-220)。W5500(A1)のVCC(+3V3_A)専用
J1 (ESP32_DevKitC_Socket_40P)	Freenove ESP32-WROOM-32E DevKitC(40pin、25.4mm幅)を直接プラグインするメスソケット
D1 (赤色LED) + R10 (10kΩ)	Power(Q5出力・スイッチ後12V)系統の通電表示。直径3mm・リードピッチ2.54mmのLEDをR10(10kΩ)で電流制限しGNDへ
D2 (緑色LED) + R11 (10kΩ)	+12V(J2入力・未スイッチ)系統の通電表示。直径3mm・リードピッチ2.54mmのLEDをR11(10kΩ)で電流制限しGNDへ

2.2 SPI配線 (ESP32 ⇄ W5500)

W5500	ESP32 GPIO
SCK	GPIO 18
MISO	GPIO 19
MOSI	GPIO 23
CS (SS)	GPIO 5
RST	GPIO 21 (active-LOW。起動時にESP32からパルスを出してハードリセット)
VCC	3.3V
GND	GND

2.3 出力ピン割り当て

チャンネル	ESP32 GPIO	論理	起動時状態	駆動回路	出力先
POWER (12V 電源)	GPIO 26	active-HIGH	OFF	R6→Q1(2SC1815)→Q5 (2SJ334, PMOSハイサイドスイッチ)	J5 (Power)
PTT	GPIO 27	active-HIGH	OFF	R7→Q3(2SC1815)	J6 (PTT_ON、無線機 PTT端子をGND側へ落とす方式)

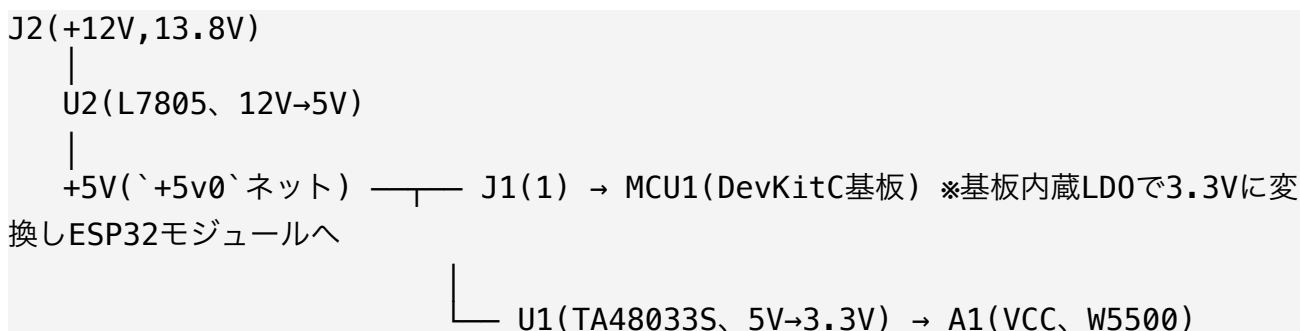
GPIO25は旧仕様(Rev.1.1)でLNA制御用として予約されていたが、実機回路では駆動回路が実装されておらず未接続。現行スケッチ・本書では扱わない。

POWER(GPIO26)・PTTともにラッチ式のON/OFF出力であり、送信/受信

の自動切替シーケンス（100ms待機等）は行わない。POWERとPTTは完全に独立したチャンネルとして扱う。

2.4 電源系統

外部から供給される+12V(13.8V)を起点に、**U2(L7805)**が生成する**+5V(+5v0)**を**MCU1 (DevKitC基板)**と**U1 (TA48033S)**の両方に分岐供給する、単一系統の構成である。



供給先	経路	備考
MCU1 (DevKitC) 用	+12V → U2(L7805、12V→5V) → J1(1)	DevKitC基板上のオンボードLDOがこの5Vを3.3Vに変換しESP32モジュールへ供給する
W5500 用	+12V → U2(L7805、12V→5V) → U1(TA48033S、5V→3.3V) → A1(VCC)	U2出力の+5v0をU1がさらに3.3Vへ降圧しW5500(A1)のVCCへ供給

MCU1用・W5500用いずれも起点はU2(L7805)の+5V出力であり、外付けのDCDCバックコンバータモジュールは使用しない（旧Rev.2.0で存在したJ3は廃止）。詳細な接続関係はdocs/MCU1_J1_W5500_接続一覧.md「電源系統の接続詳細」を参照。

3. 動作仕様

Rev.1.1までの「LNA off→100ms待機→PTT/PA on」のようなシーケンス制御は廃止し、POWERとPTTはそれぞれ独立したON/OFFイベントに連動する。

3.1 PTT (shonan-android 送信ボタン連動)

- 送信ボタンON: GET /tx?state=on → 設定した遅延時間 (ptt_delay_ms、デフォルト50ms)後にPTT(GPIO27) ON
- 送信ボタンOFF: GET /tx?state=off → PTT(GPIO27) 即時OFF (保留中のON遅延はキャンセルされる)
- 12V電源には一切触れない。

3.2 12V電源 (Shonan_lite-PI4アプリ起動/終了連動)

- アプリ起動から5秒後: GET /ch?idx=0&state=on → 設定した遅延時間 (power_delay_sec、デフォルト3秒)後にPOWER(GPIO26) ON
- アプリ終了時: 先に GET /ch?idx=0&state=off → POWER(GPIO26) 即時OFF (保留中のON遅延はキャンセルされる)、その後3秒待ってからアプリを終了

3.3 設計意図

- LNA駆動回路が実機に存在しないため、送受信切替に伴う保護シーケンス (LNA切り離し等) は不要と判断し廃止した。
- 12V電源はPA等の外部機器向けの主電源に相当するため、TX/RXの都度切り替えるのではなく、アプリの起動/終了という粒度の大きいタイミングでON/OFFする運用とする。
- PTTのみを送信ボタンに連動させることで、TX切替の応答を遅延なく行う。
- ON要求のみ設定した遅延時間を挟み、OFF要求は安全のため常に即時反映する (保留中のON遅延もOFF要求でキャンセルされる)。

4. ネットワーク・HTTP API仕様

4.1 ネットワーク設定

- 固定IPアドレス方式 (初期値 192.168.0.100/24、ゲートウェイ 192.168.0.1)。DHCPは使用しない
- IP/ゲートウェイ/サブネットはNVS(Preferences)に保存され、電源断後も保持される。/config/network (下記4.2) で変更可能で、保存後は自動再起動して新設定を反映する
- MACアドレスはスケッチ内で固定値を指定 (同一LAN内で重複しない)

こと)

4.2 API一覧

エンドポイント	メソッド	説明	レスポンス
/	GET	ステータス確認用HTML（手動ON/OFFボタン付き）	HTML
/tx?state=on	GET	PTT ON （shonan-androidから呼び出し）	TX（プレーンテキスト）
/tx?state=off	GET	PTT OFF （shonan-androidから呼び出し）	RX（プレーンテキスト）
/toggle?ch=0..1	GET	個別チャンネル手動トグル（配線確認用デバッグ機能。 0=POWER,1=PTT）	/ ヘリダイレクト
/ch?idx=0..1&state=on off	GET	個別チャンネル明示ON/OFF（0=POWER,1=PTT。 Shonan_lite-PI4 GUI起動/終了時のGPIO26制御用）	ON/OFF（プレーンテキスト）

/api/status	GET	現在状態をJSONで取得	{"power":bool,"ptt":bool,"tx_active":bool}
/config	GET	遅延時間・IP設定画面 (HTML)	HTML
/config/delay?power_delay_sec=..&ptt_delay_ms=..	GET	POWER/PTTのON遅延時間を保存	設定画面へリダイレクト等
/config/network?ip=..&gateway=..&subnet=..	GET	固定IPアドレスを保存し自動再起動	設定画面へリダイレクト等

4.3 呼び出し例

送信開始時: GET http://<ESP32のIPアドレス>/tx?state=on

送信終了時: GET http://<ESP32のIPアドレス>/tx?state=off

ESP32のIPアドレスは固定IP(初期値192.168.0.100、4.1参照)のため、shonan-android・Shonan_lite-PI4(pi4/gui)いずれも設定画面で利用者が同じIPアドレスを入力・保持する運用とする。ESP32側のIPを/config/networkで変更した場合は、アプリ側の設定も合わせて変更すること (★DDNS/mDNS等による自動検出は本版では未実装)。

4.4 Shonan_lite-PI4(pi4/gui)側の連携

- 設定画面(pi4/gui/screens/settings.py)の「PA_Power/PTTコントローラ (ESP32)」欄で「ESP32 W5500を使用する」をONにし、ESP32のIPアドレスを設定する。OFFまたは空欄の場合は連携自体を行わない (OFFにしてもIPアドレスは保持される。未接続環境でもTX/RXの動作に影響しない)。
- pi4/gui/backend.py の TxController.start() 冒頭で /tx?state=on、stop() 冒頭で /tx?state=off をGETする (PTTのみをON/OFFする)。タイムアウトは短く(1.5秒)設定し、ESP32が未接続/未応答でも例外を握りつぶしてTX本体の動作を妨げない (ログにのみ記録)。
- ESP32(MCU1)とPluto+の間に直接の通信経路はない。Pi4(pi4/gui)がPluto+へはSSHで/www/settings.txtを書き込み (_push_pluto_settings())、ESP32へはHTTPで/tx?

state=(_send_ptt_request())を送る、それぞれ独立した構成であり、ESP32はPi4からのTX開始/終了通知のみを扱う。

- ・ 上記PTT切替とは別に、Pi4アプリ(pi4/gui/main.py)自体の起動/終了に連動してGPIO26(POWERチャンネル、idx=0)を明示的に制御する(_send_ptt_channel_state()、/ch?idx=0&state=on|offを使用)。
 - アプリ起動から5秒後にGPIO26をON(Langstone V2Modifyへの切替時・起動メニューでLangstoneを選んだ時もON)
 - アプリ終了時、先にGPIO26をOFFにしてから3秒待って実際に終了
 - 12V電源の制御にPi4本体(Raspberry Pi4)のGPIOは使用しない。あくまでMCU1側のGPIO26をネットワーク経由で制御する。
- ・ ★ESP32 W5500(本コントローラ)は必須ではない。Pi4本体のGPIO21(40番ピン、GNDは39番ピン)が送信中HIGH(3.3V)・受信中LOWになるため、これをトランジスタ/リレードライバ等でバッファすればESP32なしでもPA・LNAの送受信切替ができる(Langstone V2ModifyのTx Outputと同じピン。Shonan_Lite側はpi4/gui/backend.pyの_set_pi_tx_gpio()でpinctrlにより出力)。ESP32を併用する場合は本コントローラのPTT(J6)も同時に切り替わる。12V電源のON/OFFは本コントローラが必要。

5. 未確定・今後の課題

- ・ ★ 12V電源／PTTの実際の駆動回路との結合試験（Q1/Q3/Q5の実機動作確認）は未実施。
- ・ ★ mDNS対応（<http://shonan-ptt.local/> 等）による自動検出は未実装（IPアドレスは固定IP方式で、アプリ側に手入力する）。運用上必要であれば追加検討。
- ・ ★ shonan-androidアプリ側でのHTTPリクエスト送出実装は本スケッチのスコープ外。アプリ側の送信ボタンハンドラに追加が必要。
- ・ 実機ESP32への書き込みは完了（2026-08-30、MAC: 70:4b:ca:7b:eb:94）。ただしW5500・12V電源/PTT駆動回路を実際に接続した結合試験、Pi4(pi4/gui)側との通信確認は未実施。
- ・ GPIO25(旧LNA)は物理的に未接続のまま。今後LNA制御が必要になった場合は、駆動回路の追加とスケッチ・本書の再改訂が必要。

6. 関連ファイル

- スケッチ本体: hardware/W5500_PA_PTT_Control/W5500_PA_PTT_Control.ino
- 回路図: hardware/W5500_PA_PTT_Control/kicad/w5500-esp32.kicad_sch (KiCad原本) / hardware/W5500_PA_PTT_Control/docs/W5500_PA_PTT_Control_回路図.svg (KiCadから書き出したSVG。旧Rev.1.0の手書き概略図は実機と内容が乖離していたため2026-08-30に廃止し、KiCad原本からの書き出しに置き換えた)
- MCU1/J1/W5500ピン対応の詳細表: hardware/W5500_PA_PTT_Control/docs/MCU1_J1_W5500_接続一覧.md